

SOFTWARE ENGINEERING & ACADEMIC PROJECT ARTIFACT

PlacementTracker (Study Buddy)

Software Requirements Specification (SRS)

A full-stack placement preparation tracking platform engineered specifically for MCA students at Sri Venkateswara College of Engineering (SVCE). This document provides an exhaustive, code-verified specification of all system requirements, architectural patterns, schemas, and operational constraints.

Project Title:	PlacementTracker — SVCE Placement Prep Suite
Target Audience:	SVCE MCA Students & Faculty Review Panel
System Version:	v1.0.0 (Production Release)
Primary Stack:	Next.js 14 App Router, MongoDB Atlas, NextAuth, Groq LLM Engine
Document Date:	July 2026
Verification:	100% Codebase-Verified (17 Mongoose Models & Serverless API Routes)

SOFTWARE ENGINEERING & ACADEMIC PROJECT ARTIFACT

PlacementTracker (Study Buddy)

Software Requirements Specification (SRS)

A full-stack placement preparation tracking platform engineered specifically for MCA students at Sri Venkateswara College of Engineering (SVCE). This document provides an exhaustive, code-verified specification of all system requirements, architectural patterns, schemas, and operational constraints.

Project Title:	PlacementTracker — SVCE Placement Prep Suite
Target Audience:	SVCE MCA Students & Faculty Review Panel
System Version:	v1.0.0 (Production Release)
Primary Stack:	Next.js 14 App Router, MongoDB Atlas, NextAuth, Groq LLM Engine
Document Date:	July 2026
Verification:	100% Codebase-Verified (17 Mongoose Models & Serverless API Routes)

Table of Contents

- [1. Introduction](#1-introduction)

- [1.1 Purpose](#11-purpose) - [1.2 Scope](#12-scope) - [1.3 Intended Audience](#13-intended-audience) - [1.4 Definitions and Acronyms](#14-definitions-and-acronyms)

- [2. Overall Description](#2-overall-description)

- [2.1 Product Perspective](#21-product-perspective) - [2.2 Product Features](#22-product-features) - [2.3 User Classes and Characteristics](#23-user-classes-and-characteristics) - [2.4 Operating Environment](#24-operating-environment)

- [3. System Architecture](#3-system-architecture)

- [3.1 Tech Stack Infrastructure](#31-tech-stack-infrastructure) - [3.2 High-Level Architecture Diagram](#32-high-level-architecture-diagram) - [3.3 Caching & Performance Strategy](#33-caching--performance-strategy)

- [4. Data Models & Schemas](#4-data-models--schemas)

- [4.1 Core User & Auth Models](#41-core-user--auth-models) - [4.2 DSA Tracker & Progress Models](#42-dsa-tracker--progress-models) - [4.3 Company Preparation Models](#43-company-preparation-models) - [4.4 Community & Discuss Forum Models](#44-community--discuss-forum-models) - [4.5 Logic Foundations Models](#45-logic-foundations-models) - [4.6 AI Assistant & Notification Models](#46-ai-assistant--notification-models)

- [5. Functional Requirements](#5-functional-requirements)

- [FR-1: Authentication & Session Management](#fr-1-authentication--session-management) - [FR-2: Student Onboarding & Target Setting](#fr-2-student-onboarding--target-setting) - [FR-3: DSA Progress Tracker & Tiered Grouping](#fr-3-dsa-progress-tracker--tiered-grouping) - [FR-4: Daily Activity Log & Streak Engine](#fr-4-daily-activity-log--streak-engine) - [FR-5: Student Performance Dashboard](#fr-5-student-performance-dashboard) - [FR-6: Company-Specific Placement Preparation](#fr-6-company-specific-placement-preparation) - [FR-7: Peer Leaderboard & Ranking System](#fr-7-peer-leaderboard--ranking-system) - [FR-8: Peer Discuss Forum & Q&A](#fr-8-peer-discuss-forum--qa) - [FR-9: Notification Engine & Automated Cron Reminders](#fr-9-notification-engine--automated-cron-reminders) - [FR-10: AI Study Buddy Chatbot](#fr-10-ai-study-buddy-chatbot) - [FR-11: Logic Foundations Zone](#fr-11-logic-foundations-zone) - [FR-12: Terms of Service Agreement Gate](#fr-12-terms-of-service-agreement-gate) - [FR-13: Interactive Code Console (UI Teaser / Future Scope)](#fr-13-interactive-code-console-ui-teaser--future-scope)

- [6. Non-Functional Requirements](#6-non-functional-requirements)

- [6.1 Performance Requirements](#61-performance-requirements) - [6.2 Scalability Requirements](#62-scalability-requirements) - [6.3 Security Requirements](#63-security-requirements) - [6.4 Reliability & Availability Requirements](#64-reliability--availability-requirements) - [6.5 Usability & Accessibility Requirements](#65-usability--accessibility-requirements) - [6.6 Privacy & Compliance](#66-privacy--compliance)

- [7. External Interface Requirements](#7-external-interface-requirements)

- [8. Future Scope & Roadmap](#8-future-scope--roadmap)

- [9. Conclusion](#9-conclusion)

1. Introduction

1.1 Purpose

PlacementTracker (Study Buddy) is a specialized, web-based placement preparation and progress tracking system developed specifically for Master of Computer Applications (MCA) students at Sri Venkateswara College of Engineering (SVCE).

Preparing for campus placement drives requires structured practice across Data Structures & Algorithms (DSA), company-specific problem sets, foundational programming concepts, and continuous peer benchmarking. Existing general-purpose tools lack localized tracking, peer collaboration tailored to college cohorts, and intelligent assistance. PlacementTracker solves this by delivering an integrated platform where students track DSA problem-solving counts, manage company target notes, engage in peer discussion, and receive personalized guidance from an AI assistant.

This document serves as the authoritative **Software Requirements Specification (SRS)** for PlacementTracker. It details all operational capabilities, system architecture, database schemas, functional requirements, and performance parameters verified directly against the production codebase.

1.2 Scope

PlacementTracker encompasses the following core operational modules:

- **Authentication & Onboarding:** OAuth 2.0 single sign-on via Google workspace/personal accounts, custom goal setting, and branch/year profile completion.

DSA Progress Tracker: Structured tracking across 12 canonical Data Structure and Algorithm topics categorized under four progressive learning tiers (Fundamentals, Core Structures, Trees & Graphs, and Advanced*).

- **Company Preparation Hub:** Tracking target hiring companies (e.g., TCS, Infosys, Accenture, Amazon, Zoho), setting preparation statuses (`not_started` , `in_progress` , `done`), maintaining rich markdown prep notes with autosave, and sharing preparation links.
- **Analytics Dashboard:** Visual progress representation with animated statistics, streak counters, daily goal meters, and 3D canvas backgrounds.
- **Peer Leaderboard:** Real-time student ranking engine supporting All-Time total problem counts, Weekly activity filters, and privacy-preserving Anonymous Mode.
- **Discuss Forum:** Community discussion system supporting problem-linked posts, syntax-highlighted code snippets, image attachments, upvoting, nested replies, and accepted answer markings.
- **AI Study Buddy:** An intelligent conversational chatbot powered by Groq's `llama-3.3-70b-versatile` LLM, utilizing multi-key failover rotation across 5 API keys and enforced daily query rate limits.

Logic Foundations Zone: A dedicated practice area for beginners featuring categorized logic-building problems (Warmup, Easy, Core*) with interactive hint toggles and sample test cases.

- **Notification Engine:** Real-time in-app notification system for post replies, upvotes, accepted answers, and automated daily streak warning cron jobs.
- **Terms Gate:** Mandatory Terms of Service modal flow enforcing user privacy and platform usage rules before application access.

Explicit Out-of-Scope Items

Live Code Execution Sandbox: The platform currently provides a visual UI teaser for the Code Console*. In-browser code compilation and sandboxed code execution (via Judge0/Piston) are explicitly out-of-scope for the current production build and scheduled for v2.0.

- **Official Institutional Integration:** PlacementTracker operates as an independent student-built tool and does not interface directly with SVCE's internal marks or attendance portal databases.

1.3 Intended Audience

This document is prepared for: 1. **Academic Reviewers & Evaluation Panel:** Faculty members evaluating the software engineering architecture, database design, and functional implementation for academic accreditation. 2. **Software Developers & Maintainers:** Engineers extending or maintaining the codebase, providing an exact reference for schemas, API contracts, and key rotation logic. 3. **Student Users:** SVCE MCA candidates seeking an overview of system capabilities and privacy controls.

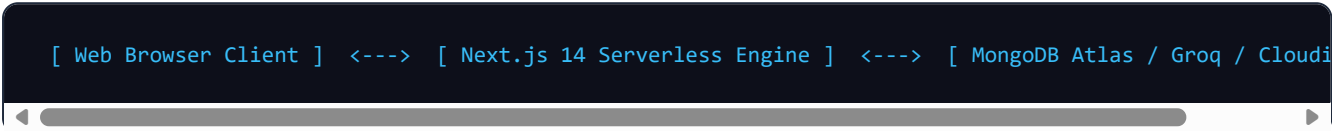
1.4 Definitions and Acronyms

Term / Acronym	Full Definition / Context
DSA	Data Structures and Algorithms — core curriculum for technical placement tests.
SRS	Software Requirements Specification — formal engineering document defining system scope.
NextAuth	Open-source authentication library for Next.js implementing OAuth 2.0 session handling.
MongoDB Atlas	Fully managed cloud database engine hosting PlacementTracker's 17 collection models.
Groq API	Ultra-fast LPU inference engine powering the AI Study Buddy with llama-3.3-70b-versatile .
ISR	Incremental Static Regeneration — Next.js rendering technique used for public profile caching.
SWR	Stale-While-Revalidate — React hooks library for client-side data fetching and optimistic UI.
Cloudinary	Cloud media storage platform managing user-uploaded discuss images via signed API signatures.
SVCE	Sri Venkateswara College of Engineering — institutional context for the MCA batch deployment.

2. Overall Description

2.1 Product Perspective

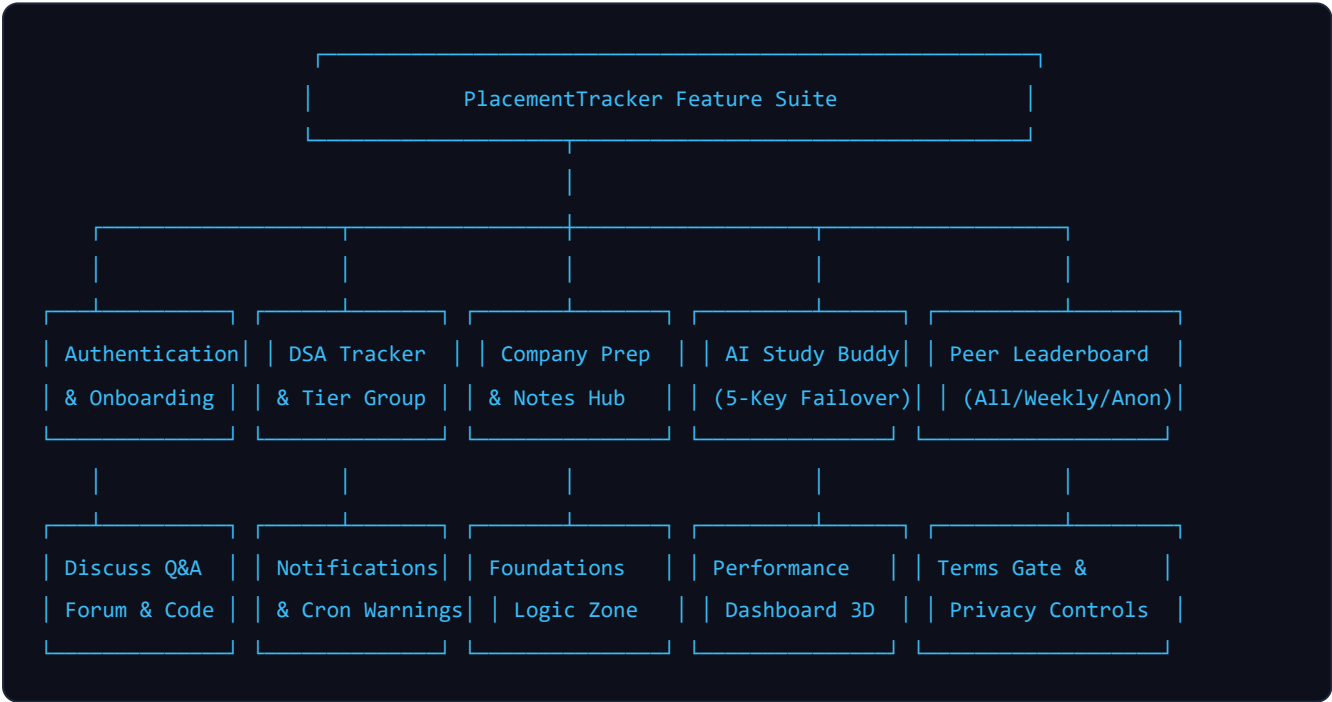
PlacementTracker is a standalone, full-stack Web Application deployed on Vercel's serverless edge infrastructure. It operates on a multi-tier client-server architecture:



The application is completely self-contained. It leverages Google OAuth 2.0 for identity verification without storing plain text passwords. Persistence is handled via MongoDB Atlas, media is stored in Cloudinary, and AI capabilities are routed through Groq's high-speed inference cloud.

2.2 Product Features

The system provides 12 primary features:



1. **Google Authentication:** Single-click login/registration via Google OAuth. 2. **Guided Onboarding:** Multi-step setup wizard for focus topic selection and daily problem targets. 3. **Tiered DSA Tracker:** Reordered topic layout categorized into 4 tiers with goal progress bars. 4. **Company Hub:** Preparation status management, resource sharing, and styled notepad notes. 5. **AI Study Buddy Widget:** Chatbot with personalized context injection, markdown support, and key rotation. 6. **Leaderboard Engine:** Real-time scoring based on solved problems, supporting anonymous toggle. 7. **Community Discuss:** Rich Q&A forum supporting syntax-highlighted code blocks and accepted answers. 8. **Foundations Zone:** Logic building problems categorized by difficulty with

sample test cases. 9. **Student Dashboard**: Visual metrics dashboard with 3D canvas, daily goals, and streak metrics. 10. **Notification Bell**: Unread indicator and list of forum interactions and automated streak warnings. 11. **Public Profiles**: Shareable public student profile links cached with Incremental Static Regeneration (ISR). 12. **Code Console Teaser**: Visual UI preview highlighting multi-language compilation roadmap.

2.3 User Classes and Characteristics

- **Authenticated Students (Primary Class)**: Logged-in SVCE MCA candidates. Possess full access to update tracker progress, write company notes, post on Discuss, query the AI Study Buddy, and customize leaderboard anonymity.
- **Public Visitors (Secondary Class)**: Unauthenticated external viewers accessing student profile links (`/profile/[userId]`). Access is restricted to read-only views of publicly shareable progress statistics.
- **System Cron Executor**: Automated service account executing daily background jobs (`/api/cron/streak-warning`) authorized via `CRON_SECRET` .

2.4 Operating Environment

PlacementTracker is designed to operate under the following environment:

- **Client Side**: Modern HTML5/CSS3 browsers (Google Chrome 100+, Mozilla Firefox 100+, Apple Safari 15+, Microsoft Edge 100+). Mobile responsive from 360px viewport widths up to 4K displays.
- **Server Side**: Node.js runtime (v18.x / v20.x) executing on Vercel Serverless Functions.
- **Database Tier**: MongoDB Atlas Cluster (v6.0+ Wire Protocol compliant).

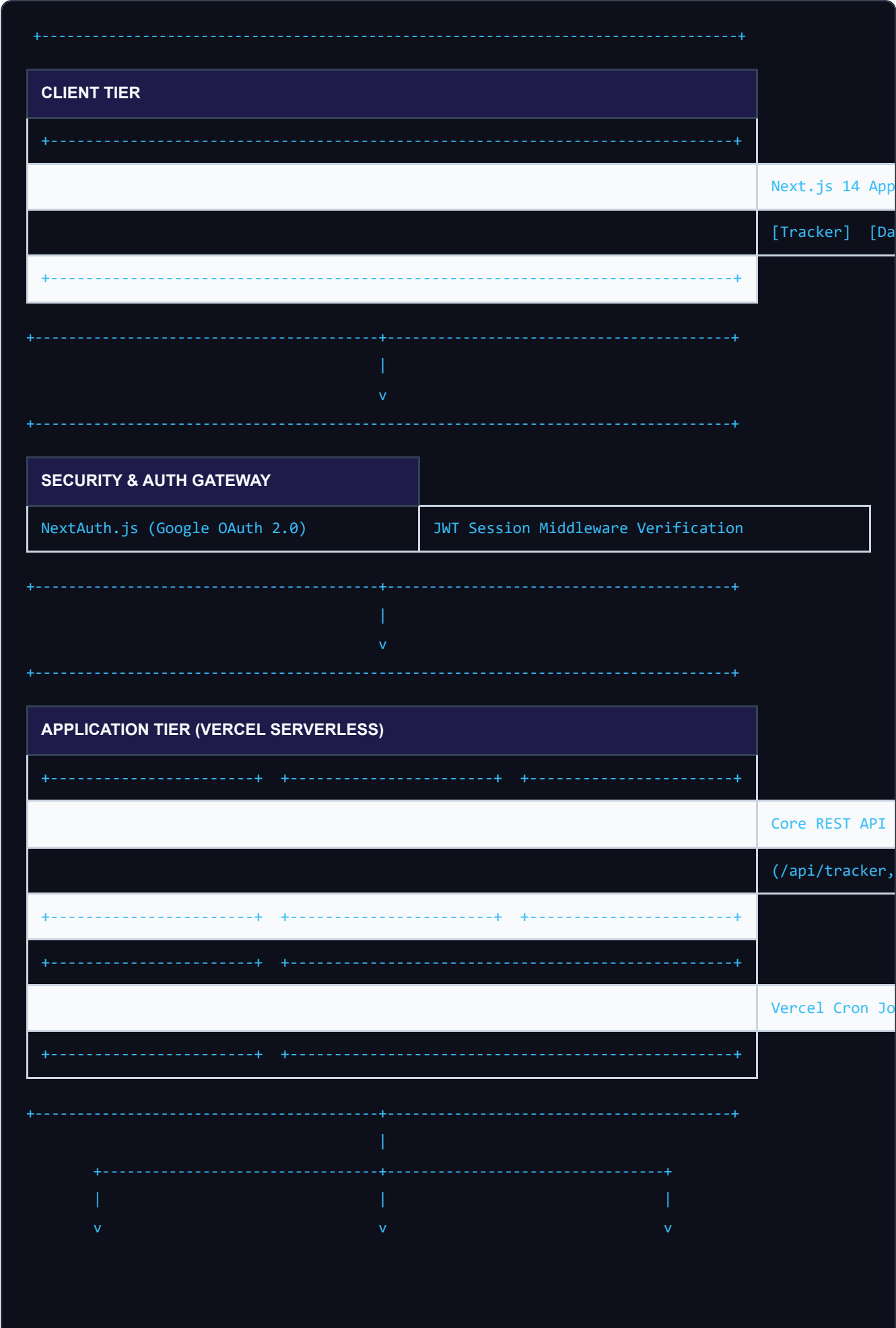
3. System Architecture

3.1 Tech Stack Infrastructure

Layer / Subsystem	Technology Selected	Version / Package	Strategic Justification
Frontend Framework	Next.js App Router	v14.2.35	Server-Side Rendering (SSR), API routes, and optimized client routing.
UI Library & Styling	React + Tailwind CSS	React 18 / v3.4.1	Utility-first responsive design, dark mode palette, and design consistency.
Motion & 3D	Framer Motion & R3F	framer-motion 12 , @react-three/fiber 8	Fluid micro-animations, layout transitions, and interactive 3D background canvas.
Data Fetching	SWR	v2.4.2	Client-side stale-while-revalidate caching, optimistic updates, and fast re-renders.
Database ORM	Mongoose	v9.7.2	Strictly typed schema enforcement and collection index orchestration.
Cloud Database	MongoDB Atlas	Cloud Cluster	Scalable document storage with connection pooling for serverless execution.
Authentication	NextAuth.js	v4.24.14	Secure JWT session management with @auth/mongodb-adapter integration.
AI Inference	Groq SDK	groq-sdk 1.3.0	Ultra-fast execution of llama-3.3-70b-versatile with custom 5-key failover.
Media Storage	Cloudinary	next-cloudinary 6.17	Cloud media uploads, auto-optimization, and signed upload signature generation.

3.2 High-Level Architecture Diagram

The system architecture follows a gated serverless model. Requests originating from the client browser pass through NextAuth middleware for authentication validation before hitting Next.js serverless API routes. API handlers interface with MongoDB Atlas via a cached connection pool handler, communicate with Groq AI using multi-key failover logic, and issue signed media upload tokens for Cloudinary.



+-----+		+-----+		+-----+	
MongoDB Atlas		Groq AI Cloud		Cloudinary CDN	
(17 Schemas)		(Llama 3.3 70B		(Media Assets)	
+-----+		+-----+		+-----+	

3.3 Caching & Performance Strategy

1. **Database Connection Pooling:** Serverless functions instantiate new environments dynamically. PlacementTracker uses a global connection cache (`global._mongooseConn` in `lib/dbConnect.ts`) to reuse active Mongoose connections across warm lambda invocations, eliminating MongoDB connection overhead. 2. **Optimistic UI Updates:** Client-side state in the DSA Tracker and Discuss Forum updates immediately upon user action, issuing background HTTP requests. If the network call fails, state cleanly rolls back with Sonner toast notifications. 3. **Public Profile ISR:** Public student profile routes (`/profile/[userId]`) utilize Next.js Incremental Static Regeneration (`export const revalidate = 60`) to serve pre-rendered HTML from Vercel's Edge Network while background-updating every 60 seconds.

4. Data Models & Schemas

PlacementTracker's database architecture consists of **17 Mongoose models**. Below are the exact, code-verified field definitions, data types, and operational purposes.

4.1 Core User & Auth Models

1. User Model (`User.ts`)

Stores core student account metadata, activity metrics, and terms acceptance state.

Field Name	Type	Options / Constraints	Operational Purpose
<code>name</code>	<code>String</code>	<code>required: true</code>	Full student name from Google OAuth profile.
<code>email</code>	<code>String</code>	<code>required: true, unique: true, index: true</code>	Primary identifier; index accelerates session lookups.
<code>photoUrl</code>	<code>String</code>	<code>default: ""</code>	Google account profile image URL.
<code>branch</code>	<code>String</code>	Optional	Academic branch (e.g., "MCA").
<code>year</code>	<code>String</code>	Optional	Academic graduation year (e.g., "2026").
<code>currentStreak</code>	<code>Number</code>	<code>default: 0</code>	Consecutive active problem-solving days.
<code>longestStreak</code>	<code>Number</code>	<code>default: 0</code>	Maximum streak milestone achieved.
<code>lastActiveDate</code>	<code>Date</code>	Optional	Timestamp of most recent problem submission.
<code>dailyGoal</code>	<code>Number</code>	<code>default: 5</code>	Student-configured daily target problem count.
<code>termsAcceptedVersion</code>	<code>String</code>	<code>default: null</code>	Version of Terms of Service accepted (e.g., "v1.0").
<code>termsAcceptedAt</code>	<code>Date</code>	<code>default: null</code>	Exact timestamp when Terms were accepted.
<code>createdAt</code>	<code>Date</code>	Mongoose timestamps	Account creation date.

2. UserOnboarding Model (`UserOnboarding.ts`)

Tracks student progress through the 3-step setup wizard.

Field Name	Type	Options / Constraints	Operational Purpose
<code>userId</code>	<code>ObjectId</code>	<code>ref: "User", required: true, unique: true</code>	Reference to owner student.
<code>isComplete</code>	<code>Boolean</code>	<code>default: false</code>	Flag indicating onboarding wizard completion.
<code>completedSteps</code>	<code>[String]</code>	Array of step strings	List of completed setup steps (e.g., <code>["topics", "goal"]</code>).
<code>dailyGoal</code>	<code>Number</code>	<code>default: 5</code>	Target setting selected during onboarding.

3. FeatureInterest Model (`FeatureInterest.ts`)

Tracks student votes for prospective features (e.g., Code Console).

Field Name	Type	Options / Constraints	Operational Purpose
<code>userId</code>	<code>ObjectId</code>	<code>ref: "User", required: true</code>	Student registering feature interest.
<code>feature</code>	<code>String</code>	<code>required: true</code>	Feature identifier string (e.g., <code>"code_console"</code>).
<code>requestedAt</code>	<code>Date</code>	<code>default: Date.now</code>	Timestamp of interest registration.

Compound Index: `{ userId: 1, feature: 1 }` (unique constraint prevents duplicate votes).

4.2 DSA Tracker & Progress Models

4. Topic Model (`Topic.ts`)

Defines the 12 canonical DSA topic modules and their resource links.

Field Name	Type	Options / Constraints	Operational Purpose
<code>name</code>	<code>String</code>	<code>required: true</code>	Topic name (e.g., "Arrays", "Dynamic Programming").
<code>order</code>	<code>Number</code>	<code>required: true</code>	Sequence order (1 to 12) defining progression sequence.

name	String	required: true	Topic name (e.g., "Arrays", "Dynamic Programming").
tier	String	enum: ["fundamentals", "core", "trees-graphs", "advanced"]	Learning progression tier grouping.
resourceLinks	[Subdoc]	{ title: String, url: String }[]	External reference tutorial links (Striver, NeetCode).

5. UserTopicProgress Model (`UserTopicProgress.ts`)

Tracks an individual student's solved problem count for a given topic.

Field Name	Type	Options / Constraints	Operational Purpose
userId	ObjectId	ref: "User", required: true	Reference to student.
topicId	ObjectId	ref: "Topic", required: true	Reference to target DSA topic.
problemsSolved	Number	default: 0	Total problems logged under this topic.
lastUpdated	Date	default: Date.now	Timestamp of last increment/exact update.

Compound Index: { `userId: 1, topicId: 1` } (unique constraint per user-topic pair).

6. DailyActivityLog Model (`DailyActivityLog.ts`)

Records daily problem-solving volume for streak calculations and activity graphs.

Field Name	Type	Options / Constraints	Operational Purpose
userId	ObjectId	ref: "User", required: true	Reference to student.
date	Date	required: true	Normalized midnight UTC date object.
problemsSolvedThatDay	Number	default: 0	Total problems logged on this specific calendar date.

Compound Index: { `userId: 1, date: 1` } (unique constraint per user-date pair).

4.3 Company Preparation Models

7. Company Model (`Company.ts`)

Master directory of campus recruitment target companies.

Field Name	Type	Options / Constraints	Operational Purpose
------------	------	-----------------------	---------------------

name	String	required: true	Company name (e.g., "TCS", "Accenture", "Amazon").
isCustom	Boolean	default: false	Flag indicating user-added custom target company.
createdBy	ObjectId	ref: "User", default: null	Creator user ID if custom company.

8. UserCompanyPrep Model (`UserCompanyPrep.ts`)

Tracks student preparation status and private rich notes for a company.

Field Name	Type	Options / Constraints	Operational Purpose
userId	ObjectId	ref: "User", required: true	Reference to student.
companyId	ObjectId	ref: "Company", required: true	Reference to target company.
status	String	enum: ["not_started", "in_progress", "done"]	Student prep status.
notes	String	default: ""	Custom markdown preparation notes.
lastUpdated	Date	default: Date.now	Timestamp of last note/status modification.

Compound Index: { `userId: 1, companyId: 1` } (unique constraint per user-company pair).

9. CompanyResource Model (`CompanyResource.ts`)

Crowdsourced study resources linked to specific companies.

Field Name	Type	Options / Constraints	Operational Purpose
companyId	ObjectId	ref: "Company", required: true	Target company reference.
addedBy	ObjectId	ref: "User", required: true	Student who posted the link.
title	String	required: true	Resource title (e.g., "TCS NQT Past Papers").
url	String	required: true	External web URL.
createdAt	Date	Mongoose timestamps	Creation timestamp.

4.4 Community & Discuss Forum Models

10. DiscussPost Model (`DiscussPost.ts`)

Primary peer forum discussion posts linked to DSA topics.

Field Name	Type	Options / Constraints	Operational Purpose
authorId	ObjectId	ref: "User", required: true	Post author student ID.
topicId	ObjectId	ref: "Topic", required: true	Associated DSA topic.
problemTitle	String	required: true	Title of problem being discussed.
problemUrl	String	Optional	External LeetCode/GFG problem link.
title	String	required: true	Forum post title.
body	String	required: true	Main discussion post text/code.
images	[Subdoc]	{ url: String, cloudinaryPublicId: String }[]	Uploaded Cloudinary image attachments.
upvotes	Number	default: 0	Total net upvote score.
upvotedBy	[ObjectId]	ref: "User"	List of user IDs who upvoted this post.

Indexes: { createdAt: -1 }, { topicId: 1, createdAt: -1 }, { upvotes: -1 }.

11. DiscussReply Model (DiscussReply.ts)

Replies and answers submitted under discuss posts.

Field Name	Type	Options / Constraints	Operational Purpose
postId	ObjectId	ref: "DiscussPost", required: true	Parent discuss post reference.
parentReplyId	ObjectId	ref: "DiscussReply", default: null	Parent reply reference for nested threads.
authorId	ObjectId	ref: "User", required: true	Reply author student ID.
body	String	required: true	Reply markdown content.
images	[Subdoc]	{ url: String, cloudinaryPublicId: String }[]	Reply image attachments.
upvotes	Number	default: 0	Reply net upvote count.
upvotedBy	[ObjectId]	ref: "User"	List of user IDs who upvoted this reply.
isAccepted	Boolean	default: false	Flag set by post author marking accepted solution.

4.5 Logic Foundations Models

12. FoundationCategory Model (`FoundationCategory.ts`)

Categories for beginners' logic building modules.

Field Name	Type	Options / Constraints	Operational Purpose
<code>name</code>	<code>String</code>	<code>required: true, unique: true</code>	Module name (e.g., "Pattern Printing", "Loops").
<code>order</code>	<code>Number</code>	<code>required: true, index: true</code>	Display order sequence.
<code>description</code>	<code>String</code>	<code>required: true</code>	Short module summary.
<code>icon</code>	<code>String</code>	<code>required: true</code>	Lucide icon identifier key.

13. FoundationProblem Model (`FoundationProblem.ts`)

Individual practice problems inside logic foundation categories.

Field Name	Type	Options / Constraints	Operational Purpose
<code>categoryId</code>	<code>ObjectId</code>	<code>ref: "FoundationCategory", required: true</code>	Parent category reference.
<code>title</code>	<code>String</code>	<code>required: true</code>	Problem title.
<code>difficulty</code>	<code>String</code>	<code>enum: ["warmup", "easy", "core"], required: true</code>	Difficulty classification tier.
<code>statement</code>	<code>String</code>	<code>required: true</code>	Problem description and rules.
<code>approachHint</code>	<code>String</code>	<code>required: true</code>	Step-by-step logic hint.
<code>sampleInput</code>	<code>String</code>	<code>required: true</code>	Input example.
<code>sampleOutput</code>	<code>String</code>	<code>required: true</code>	Expected output string.
<code>order</code>	<code>Number</code>	<code>required: true</code>	Order index inside category.

14. UserFoundationProgress Model (`UserFoundationProgress.ts`)

Completion records for foundation problems.

Field Name	Type	Options / Constraints	Operational Purpose
------------	------	-----------------------	---------------------

userId	ObjectId	ref: "User", required: true	Student reference.
problemId	ObjectId	ref: "FoundationProblem", required: true	Foundation problem reference.
completed	Boolean	default: false	Problem completion flag.
completedAt	Date	Optional	Completion timestamp.

Compound Index: { userId: 1, problemId: 1 } (unique constraint).

4.6 AI Assistant & Notification Models

15. AIChatMessage Model (AIChatMessage.ts)

Stores persistent chat history between students and the AI Study Buddy.

Field Name	Type	Options / Constraints	Operational Purpose
userId	ObjectId	ref: "User", required: true	Owner student reference.
role	String	enum: ["user", "assistant"], required: true	Message sender role.
content	String	required: true	Chat message text content.
createdAt	Date	Mongoose timestamps	Message creation timestamp.

Index: { userId: 1, createdAt: -1 } (accelerates conversation history retrieval).

16. AIChatUsage Model (AIChatUsage.ts)

Tracks daily query volume per student to enforce the 15 questions/day quota.

Field Name	Type	Options / Constraints	Operational Purpose
userId	ObjectId	ref: "User", required: true	Student reference.
date	String	required: true	Normalized date string ("YYYY-MM-DD").
questionsAsked	Number	default: 0	Counter tracking questions sent today.

Compound Index: { userId: 1, date: 1 } (unique constraint per student-date).

17. Notification Model (Notification.ts)

In-app alert items triggered by community interactions and automated cron reminders.

Field Name	Type	Options / Constraints	Operational Purpose
------------	------	-----------------------	---------------------

<code>userId</code>	<code>ObjectId</code>	<code>ref: "User", required: true</code>	Target recipient student ID.
<code>type</code>	<code>String</code>	<code>enum: ["reply_on_post", "reply_on_reply", "upvote_post", "upvote_reply", "accepted_answer", "streak_warning"]</code>	Categorized notification trigger type.
<code>fromUserId</code>	<code>ObjectId</code>	<code>ref: "User", default: null</code>	User who triggered the alert (null if system cron).
<code>referenceId</code>	<code>ObjectId</code>	<code>default: null</code>	Target post or reply ID for navigation.
<code>message</code>	<code>String</code>	<code>required: true</code>	Formatted notification text.
<code>isRead</code>	<code>Boolean</code>	<code>default: false</code>	Read status flag for top-nav bell badge count.
<code>createdAt</code>	<code>Date</code>	Mongoose timestamps	Alert creation timestamp.

Indexes: `{ userId: 1, isRead: 1 }` (polled every 60s for badge), `{ userId: 1, createdAt: -1 }`.

5. Functional Requirements

FR-1: Authentication & Session Management

- **FR-1.1:** The system shall authenticate users exclusively via Google OAuth 2.0 through NextAuth.js.
- **FR-1.2:** Upon successful authentication, the system shall verify if a corresponding `User` record exists in MongoDB. If absent, the system shall automatically initialize a new `User` document populated with profile metadata (`name` , `email` , `photoUrl`).
- **FR-1.3:** The system shall enforce JWT session security across all protected routes via Next.js middleware. Unauthenticated requests attempting to access `/dashboard` , `/tracker` , `/companies` , `/discuss` , `/leaderboard` , or `/foundations` shall be redirected to the sign-in landing view.

FR-2: Student Onboarding & Target Setting

- **FR-2.1:** New users with incomplete onboarding records (`UserOnboarding.isComplete === false`) shall be routed to the 3-step Onboarding Wizard (`/onboarding`).
- **FR-2.2:** Step 1 shall allow students to select focus DSA topics from the 12 master topics.
- **FR-2.3:** Step 2 shall allow students to configure a daily problem goal using an interactive range slider (1 to 20 problems/day; default: 5).
- **FR-2.4:** Step 3 shall display the student's initial leaderboard placement rank and complete the onboarding status.

FR-3: DSA Progress Tracker & Tiered Grouping

- **FR-3.1:** The system shall display the 12 DSA topics sorted strictly by their `order` index (1 to 12) and visually grouped into four distinct learning progression tiers:

1. **Tier 1 — Fundamentals:** *Arrays, Strings, Hashing, Sorting & Searching.* 2. **Tier 2 — Core Structures:** *Linked List, Stacks & Queues, Recursion & Backtracking.* 3. **Tier 3 — Trees & Graphs:** *Trees, Graphs.* 4. **Tier 4 — Advanced:** *Greedy, Bit Manipulation, Dynamic Programming.* **FR-3.2:** *Each topic card shall display the student's solved count, a progress bar relative to the target goal (30 problems soft target), curated external video/tutorial resource links, and collapsible practice problem lists categorized by difficulty (Easy, Medium, Hard) and platform (LeetCode, GFG*).*

- **FR-3.3:** The system shall support both single-click increment (`+1`) and exact numerical input overrides for updating topic progress via `PATCH /api/tracker/[topicId]` .
- **FR-3.4:** For users with under 10 total solved problems, the system shall render a dismissible beginner guidance hint recommending sequential progress through Tier 1.

FR-4: Daily Activity Log & Streak Engine

- **FR-4.1:** Whenever a student increments or updates topic progress, the system shall record or update a `DailyActivityLog` entry for the current date.
- **FR-4.2:** The system shall compute `currentStreak` by checking consecutive daily activity logs working backward from today. If activity occurred today or yesterday, the streak remains active; if inactive for over 48 hours, `currentStreak` resets to 0.
- **FR-4.3:** `longestStreak` shall automatically update whenever `currentStreak` exceeds the historic maximum.

FR-5: Student Performance Dashboard

FR-5.1: The dashboard (`/dashboard`) shall render four animated stat cards: Total Solved, Current Streak, Topics Touched, and Companies Prepping*.

- **FR-5.2:** The system shall display Today's Goal progress meter comparing today's logged count against `User.dailyGoal`.
- **FR-5.3:** An interactive Recharts bar chart shall visualize problem distribution across the 12 topics sorted by tier order.
- **FR-5.4:** The dashboard header shall render a dynamic Three.js canvas background generating procedural wave animations.

FR-6: Company-Specific Placement Preparation

- **FR-6.1:** The system shall maintain a master list of target companies (`/companies`) and allow students to filter by prep status (`not_started` , `in_progress` , `done`).
- **FR-6.2:** Students shall be able to add custom companies via `POST /api/companies` (`isCustom: true`).
- **FR-6.3:** Expanding a company card shall reveal a custom notepad interface allowing markdown prep notes with real-time character count, auto-save status indicator, and explicit Save/Discard controls.
- **FR-6.4:** Students shall be able to share crowdsourced resource links for any company (`POST /api/companies/[companyId]/resources`).

FR-7: Peer Leaderboard & Ranking System

- **FR-7.1:** The system shall aggregate total solved problem counts across all students and render a ranked leaderboard (`/leaderboard`).
- **FR-7.2:** The system shall support two view modes: **All-Time** (lifetime total solved) and **Weekly** (problems logged in the trailing 7 days).
- **FR-7.3:** Top 3 ranks shall feature distinct podium badges (Gold 🏆 , Silver 🥈 , Bronze 🥉).
- **FR-7.4:** The system shall provide an **Anonymous Mode** toggle (`POST /api/leaderboard/anonymous`). When enabled, the user's name is rendered as "Anonymous Student" and their avatar is hidden on public rankings.

FR-8: Peer Discuss Forum & Q&A

FR-8.1: The system shall support community discussions (`/discuss`) filtered by topic and sorted by Latest, Most Upvoted, or Unanswered*.

- **FR-8.2:** Posts shall include problem title, problem URL, post title, markdown body, code blocks with syntax highlighting, and optional Cloudinary image attachments.
- **FR-8.3:** Authenticated users shall be able to upvote posts and replies. Duplicate upvotes toggle the upvote off.
- **FR-8.4:** Post authors shall have sole authority to mark a reply as the **Accepted Answer** (`PATCH /api/discuss/[postId]/reply/[replyId]/accept`).

FR-9: Notification Engine & Automated Cron Reminders

- **FR-9.1:** The system shall create in-app notifications (`Notification` collection) when a user receives post replies, reply upvotes, or accepted answer markings.
- **FR-9.2:** The top navigation bar shall render a notification bell icon polling unread count every 60 seconds (`GET /api/notifications/unread-count`).
- **FR-9.3:** A daily Vercel Cron job (`/api/cron/streak-warning`) authorized via `CRON_SECRET` shall query users at risk of losing an active streak (>24h inactive) and auto-generate `streak_warning` notifications.

FR-10: AI Study Buddy Chatbot

- **FR-10.1:** The system shall provide a floating AI Study Buddy chat widget accessible across all pages.
- **FR-10.2:** The API route (`POST /api/assistant`) shall construct a system context block containing the user's real name, streak, total solved count, topic breakdown, company notes, and 7-day activity.
- **FR-10.3:** To prevent AI API failures, `askAssistant()` in `lib/ai.ts` shall execute multi-key rotation across **5 Groq API keys** (`GROQ_API_KEY_1` to `5`). Upon encountering an HTTP 429 rate limit error, the system shall seamlessly retry on the next key.
- **FR-10.4:** The system shall enforce a strict daily rate limit of **15 questions/day** per user tracked via `AIChatUsage` .
- **FR-10.5:** Chat history shall persist in `AIChatMessage` . A **Clear Chat** button in the widget header shall invoke `DELETE /api/assistant/history` to wipe persistent chat history.

FR-11: Logic Foundations Zone

FR-11.1: The system shall provide a Logic Foundations zone (`/foundations`) for beginners featuring logic building modules (Pattern Printing, Basic Math, Array Logic*). **FR-11.2:** Problems shall be classified into Warmup, Easy, and Core* difficulties, complete with collapsible step-by-step logic hints, sample inputs, and expected outputs.

FR-12: Terms of Service Agreement Gate

- **FR-12.1:** The system shall render a mandatory Terms of Service modal gate whenever an authenticated user has `termsAcceptedVersion === null`.
- **FR-12.2:** The user must review platform privacy terms and click "I Agree" (`POST /api/user/accept-terms`), recording `termsAcceptedVersion: "v1.0"` and timestamp before proceeding to main application views.

FR-13: Interactive Code Console (UI Teaser / Future Scope)

FR-13.1: The system shall render an interactive Code Console UI card (`/code-console`) showcasing a mock code editor interface with language selection tabs (C++, Java, Python, JavaScript*).

- **FR-13.2:** A floating modal feature interest registration prompt shall capture user clicks on "Notify Me When Live" (`POST /api/features/interest`), saving records to the `FeatureInterest` collection.

6. Non-Functional Requirements

6.1 Performance Requirements

- **Database Query Optimization:** Critical user session lookups utilize indexed fields (`User.email` , `DiscussPost.createdAt` , `Notification.userId + isRead`).
- **Serverless Connection Reuse:** The Mongoose singleton connection wrapper prevents connection latency overhead during cold lambda invocations.
- **Client Response Time:** UI state mutations execute optimistically within 50ms, maintaining 60fps animations.

6.2 Scalability Requirements

- **Stateless Execution:** All Next.js API routes are completely stateless, enabling automatic horizontal scaling on Vercel's serverless edge infrastructure.
- **Database Aggregation:** Leaderboard queries utilize MongoDB aggregation pipelines (`$group` , `$sort` , `$limit`) capable of handling thousands of activity records efficiently.

6.3 Security Requirements

- **OAuth Session Token Security:** Session management relies entirely on encrypted HTTP-only JWT cookies managed by NextAuth.js.
- **Server-Side Environment Protection:** Database credentials (`MONGODB_URI`), Groq API keys (`GROQ_API_KEY_1..5`), and Cloudinary secrets are strictly restricted to server-side execution and never exposed to the client bundle.
- **Cron Route Protection:** The automated background endpoint (`/api/cron/streak-warning`) requires a valid `Authorization: Bearer` header.

6.4 Reliability & Availability Requirements

- **Groq Multi-Key Rotation:** The AI subsystem tolerates rate-limit spikes on individual free-tier keys by automatically rotating across 5 configured keys before reporting an error.
- **Graceful API Degradation:** Network failures during API calls trigger non-blocking toast notifications via Sonner, preserving local component state.

6.5 Usability & Accessibility Requirements

- **Responsive Layout:** Fully adaptive layouts utilizing Tailwind CSS grid and flexbox patterns, supporting screen resolutions down to 360px width.
- **Visual Feedback:** Comprehensive use of skeleton loading cards, button spinner states, and high-contrast badge indicators for optimal legibility.

6.6 Privacy & Compliance

- **Data Boundary Control:** Private user assets (such as personal company notes and daily activity logs) are strictly filtered by session `userId` and never exposed on public endpoints.
- **Terms Agreement Tracking:** Mandatory acceptance records (`termsAcceptedAt` , `termsAcceptedVersion`) ensure compliance with student privacy expectations.

7. External Interface Requirements

1. **Google Identity Provider (OAuth 2.0)** Protocol*: OAuth 2.0 Authorization Code Flow. Interface*: NextAuth `GoogleProvider` using `GOOGLE_CLIENT_ID` and `GOOGLE_CLIENT_SECRET`. 2. **MongoDB Atlas Cloud Database API** Protocol*: MongoDB Wire Protocol via `mongoose`. Interface*: Connection string configured via `MONGODB_URI`. 3. **Groq AI Inference Cloud** Protocol*: HTTPS REST API via official `groq-sdk`. Interface*: Model `llama-3.3-70b-versatile` authenticated via `GROQ_API_KEY_1` through `5`. 4. **Cloudinary Media Services** Protocol*: HTTPS Signed Upload API via `next-cloudinary`. Interface*: Authenticated using `CLOUDINARY_API_KEY`, `CLOUDINARY_API_SECRET`, and `NEXT_PUBLIC_CLOUDINARY_CLOUD_NAME`. 5. **Vercel Cron Infrastructure** Protocol*: HTTPS Scheduled GET request. Interface*: Bearer authentication validated against `CRON_SECRET`.

8. Future Scope & Roadmap

PLACEMENTTRACKER v2.0 ROADMAP

1. Sandboxed Multi-Language Code Execution Engine (Judge0 / Piston Integration)

2. Real-Time AI Mock Technical Interview Simulator with Audio Synthesis

3. Company Placement Placement Analytics & Placement Drive Calendar Export

4. Automated Resume Builder & Skill Gap Analysis Engine for SVCE MCA Cohorts

1. **Sandboxed Multi-Language Code Execution Engine:** Transitioning the *Code Console* from a visual UI teaser into a functional live execution sandbox supporting C, C++, Java, and Python compilation via Judge0 or Piston APIs. 2. **AI Mock Interview Simulator:** Enhancing Study Buddy to conduct interactive, voice-assisted technical mock interviews tailored to student company targets. 3. **Placement Analytics Export:** Providing PDF report generation for students to export their DSA preparation progress for faculty reviews.

9. Conclusion

PlacementTracker (Study Buddy) provides a robust, institutional-grade placement preparation suite engineered specifically for SVCE MCA candidates. By combining structured DSA progression, company-specific preparation tracking, community discussion, and an intelligent 5-key failover AI Study Buddy, the platform bridges the gap between individual practice and collaborative success.

This Software Requirements Specification accurately reflects the production implementation across all 17 database schemas, serverless API routes, and client components, establishing a solid foundation for academic evaluation and future platform enhancements.